

Magnitude–direction decoupling versus its initialization

Research note: attributing the advantage of MD training

2026-09-21. Interim: 38 of 42 runs finished; in-progress cells are marked and will be updated when the remaining runs complete.

1 Purpose and summary of conclusions

Magnitude–direction (MD) decoupled training changes two things at once: it imposes a specific initialization, and it replaces the optimizer step with a norm-constrained one. This note separates the two, to determine whether the advantage of MD training over ordinary AdamW comes from the decoupled optimizer itself or from the initialization it happens to impose.

The comparison is between full MD training and a new regime, `md_init`, which starts from exactly the same initialization but then trains with the ordinary AdamW. Each bottleneck family is run under full MD and under `md_init` with weight decay 0.1 and 0.01, over a grid of (K, J) configurations at fixed temperature.

The main conclusions from the completed runs are:

- **Where training converges, the initialization accounts for most of the final quality.** In every pair in which both regimes reached 20k steps, `md_init` lands within 0.001–0.041 nats of full MD. Against the better weight decay (0.1) the gap is 0.001–0.025 nats. The gap favors full MD in every comparable pair, in all three families.
- **The decoupled optimizer itself contributes a stability margin.** Full MD diverged only in rows in which every regime diverged ($K = 128$ at $T = 1$, both surrogate families). `md_init` additionally diverged in six cells in which full MD converged. Where an entire row diverges, the order is systematic: the $wd = 0.01$ run dies first, $wd = 0.1$ second, and full MD last.
- **Within `md_init`, weight decay 0.1 dominates 0.01.** It reaches a lower final validation CE in every comparable pair and diverges less often (two extra divergences versus four).
- **Attribution.** The advantage of MD training decomposes into a large initialization component, which ordinary AdamW inherits by starting from the same weights, and a genuine optimizer component, which appears as a small but consistent residual quality gap ($\lesssim 0.03$ nats at the better weight decay) and, more prominently, as protection against divergence at aggressive (K, J, T) configurations.

2 The two regimes

Full MD (`model.decouple=true`). Every matrix weight is optimized as a fixed-Frobenius-norm direction with learnable per-row and per-column softplus gains; embedding rows are re-projected to unit L2 norm after every step; no parameter receives weight decay. The initialization draws every matrix entrywise $\mathcal{N}(0, 1/d_{\text{model}})$ and projects it to Frobenius norm $c_F = \sqrt{d_{\text{out}}d_{\text{in}}/d_{\text{model}}}$; embedding rows are drawn Gaussian and projected to unit L2 norm, with a fixed $\sqrt{d_{\text{model}}}$ upscale of the embedding output in the forward pass.

MD init (`model.md_init=true`, added for this comparison). The identical initialization and the identical $\sqrt{d}$ forward upscale, followed by ordinary AdamW: no gains, no re-projection after the step, and the configured weight decay applies to every ≥ 2 -D weight, embeddings included. Weight decay 0.1 matches this project’s non-MD baseline runs; 0.01 probes the sensitivity to that choice.

Initialization identity. At equal seed the two regimes were verified to start from bitwise-identical parameters at full size (all 66 parameter tensors of the 8×768 model with the bottleneck spliced in, checked in all three families) and to compute exactly equal forward logits at initialization. Unit tests additionally verify that one optimizer step leaves the norm constraints under `md_init` and preserves them under full MD. The comparison therefore isolates the optimizer step; nothing else differs.

3 Setup

All runs use the 8×768 architecture with tied embeddings, RoPE, `abs_topk` selection over $N = 1536$ bottleneck features, `residual_out` placement, batch 96×512 tokens, lr 6×10^{-4} cosine with 500-step warmup, bf16, 20k steps, and one shared seed, mirroring the earlier `uk` campaign configurations. Three bottleneck families are compared:

- **Basic LapSum** (`rou_t_soft`): hard Top- K forward with the mass-constrained LapSum surrogate at constant absolute T ;
- **RBLapSum through_rank_kappa** (T fixed, boundary floor $b_0 = 0.1$): the fully modified surrogate;
- **Hard AbsTopK**: no surrogate gradient (active features only).

The grid is $(K, J) \in \{(32, 32), (32, 480), (128, 128), (128, 384)\}$ at $T = 1$ for the two surrogate families and $K \in \{32, 512\}$ for the hard family. Because every $K = 128$, $T = 1$ configuration diverged in both surrogate families under every regime, the two $K = 128$ pairs were rerun at $T = 2$, where all regimes can converge; these rows carry an explicit $T = 2$ label. In total 42 runs.

A run is stopped and recorded as diverged when its training CE becomes non-finite, stays above its own best value plus 2.5 nats for 800 consecutive steps, or stays above 7.0 for 300 steps (after step 1500). The 800-step window is long enough that a burst-and-recover excursion does not trigger it; the one observed recovering-burst run (`rou_t_soft`, $K=32$, $J=32$, $wd=0.01$) survived several such excursions before a terminal one at step 16,700.

4 Results: basic LapSum

(K, J)	T	MD	<code>md_init</code> , $wd=0.1$	<code>md_init</code> , $wd=0.01$
(32, 32)	1	1.5497	1.5751	diverged (16,700)
(32, 480)	1	1.5740	1.5748	1.5868
(128, 128)	1	diverged (2140)	diverged (1700)	diverged (1540)
(128, 128)	2	1.4770	1.4904	1.5059
(128, 384)	1	1.4712	diverged (5300)	diverged (2080)
(128, 384)	2	1.4872	1.4978	1.5086

Final validation CE after 20k steps, or the step at which the divergence guard stopped the run.

Two observations. First, in the four rows in which both regimes converge, the `md_init` curves track the MD curves closely throughout training; the final gap to MD is 0.0008–0.025 nats at $wd=0.1$ and 0.013–0.029 at $wd=0.01$. Second, the (128, 384), $T=1$ row separates the regimes sharply: full MD converges to the best value in the family, while both `md_init` runs diverge.

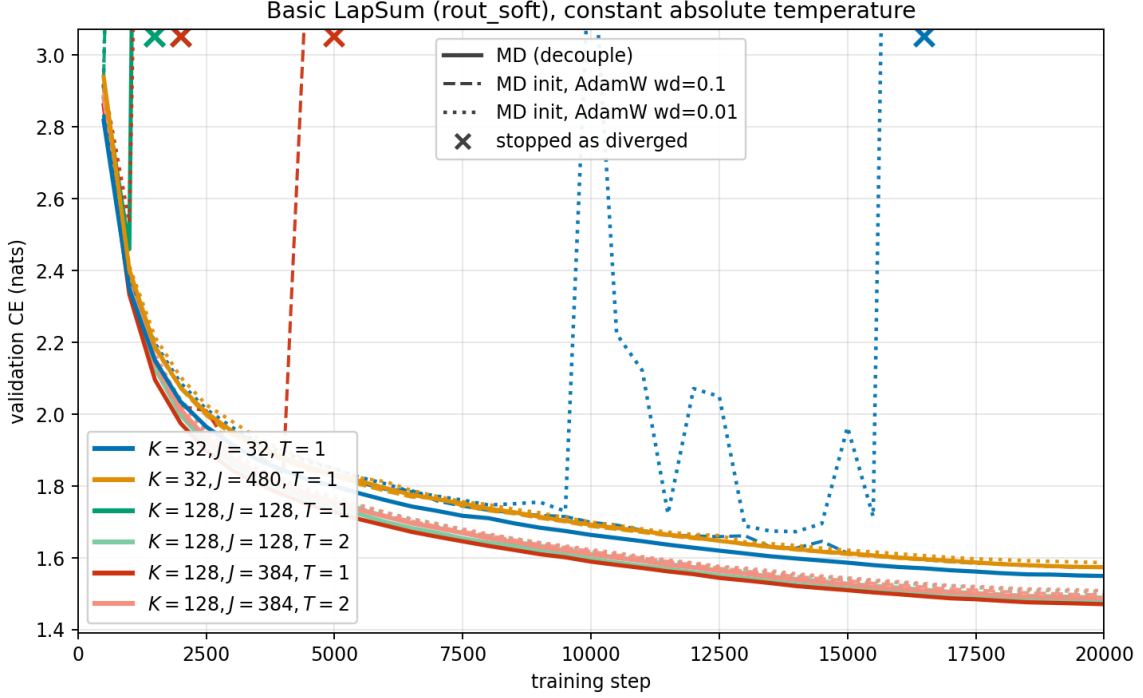


Figure 1: Validation CE for the basic LapSum family. Color encodes the (K, J, T) configuration; line style encodes the regime (full MD solid, `md_init` $wd=0.1$ dashed, $wd=0.01$ dotted). A cross marks the point at which a run was stopped as diverged; crosses at the top edge belong to curves that left the plotted range.

5 Results: RBLapSum through_rank_kappa

(K, J)	T	MD	<code>md_init</code> , $wd=0.1$	<code>md_init</code> , $wd=0.01$
(32, 32)	1	1.5482	diverged (13,860)	1.5833
(32, 480)	1	1.4923	1.5155	diverged (5000)
(128, 128)	1	diverged (1760)	diverged (1720)	diverged (1540)
(128, 128)	2	1.4778	1.538 at 14k (running)	diverged (6460)
(128, 384)	1	diverged (1880)	diverged (1800)	diverged (1640)
(128, 384)	2	1.590 at 9.5k (running)	1.748 at 4.5k (running)	1.826 at 3.5k (running)

The same pattern with more divergences, consistent with this family’s known narrower stability region at $T = 1$. Full MD again diverges only in the two rows in which everything diverges. `md_init` additionally loses (32, 32) at $wd=0.1$ (a late death at step 13,860 after repeated recovering bursts), (32, 480) at $wd=0.01$, and (128, 128), $T=2$ at $wd=0.01$; in the surviving comparable pairs the final gap to MD is 0.023–0.035 nats.

6 Results: hard AbsTopK

K	MD	<code>md_init</code> , $wd=0.1$	<code>md_init</code> , $wd=0.01$
32	1.6162	1.6375	1.6569
512	1.4298	1.4432	1.4605

With no surrogate gradient there are no divergences, and the comparison is purely about quality. The ordering $MD < wd=0.1 < wd=0.01$ holds at both K , with gaps to MD of 0.013–0.021 nats at $wd=0.1$ and 0.031–0.041 at $wd=0.01$.

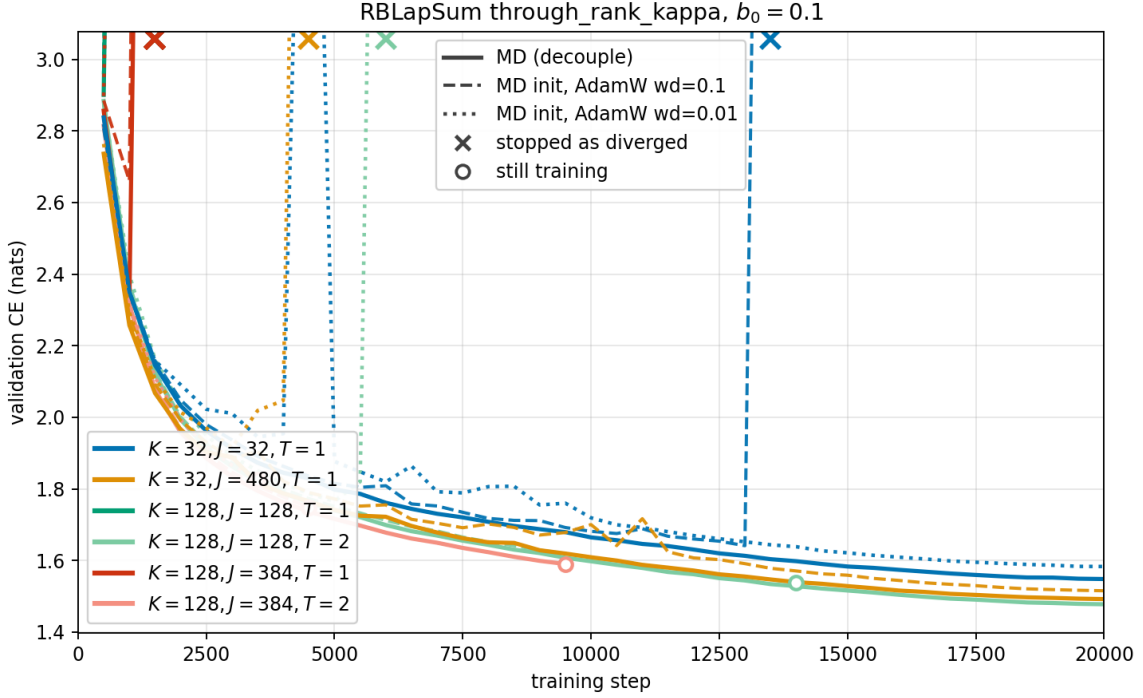


Figure 2: Validation CE for the RBLapSum `through_rank_kappa` family. Encoding as in the previous figure; an open circle marks the last available point of a run still training at the time of this snapshot.

7 Conclusions

1. **The initialization carries most of the converged quality.** Ordinary AdamW started from the MD initialization reaches within 0.001–0.025 nats of full MD at $wd=0.1$ in every configuration in which both converge, across all three families. If MD training is adopted for its final loss on stable configurations, most of that benefit is available from the initialization (with its $\sqrt{d}$ embedding upscale) alone.
2. **The decoupling itself is what buys stability.** Every configuration lost by full MD was lost by all three regimes, whereas `md_init` diverged in six additional cells spread over both surrogate families, both (K, J) extremes, and both weight decays. In rows in which all three regimes diverge, the death order is systematically $wd=0.01$ first, $wd=0.1$ second, MD last. The norm constraint does not eliminate the $K = 128, T = 1$ instability, but it consistently delays it and widens the set of configurations that survive 20k steps.
3. **Full MD retains a small, consistent quality edge** of 0.001–0.041 nats over `md_init` in every completed pair. The edge grows with the weaker weight decay, so part of it is attributable to the regularization difference rather than to the geometry of the update.
4. **Scope.** All cells are single runs at one seed. Earlier work in this project showed that divergence at marginal configurations is stochastic, so individual cell outcomes should be read as draws from a divergence rate; the row-level patterns (all-regime deaths at $K=128, T=1$; the systematic death ordering; MD never diverging alone) are the robust part. Four runs (marked “running”) had not finished at the time of writing; the tables and figures will be updated when they complete.

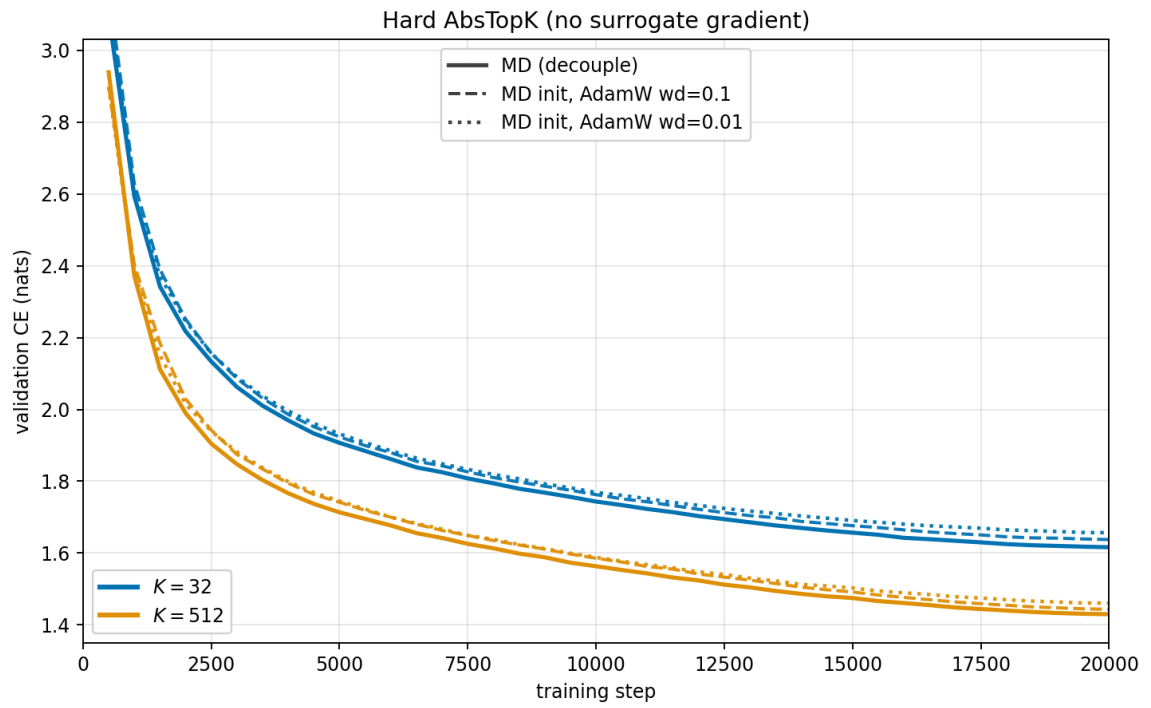


Figure 3: Validation CE for the hard AbsTopK family (no surrogate gradient). Encoding as in the previous figures. No run in this family diverged.